



umwl-tui — del export de flujos de Illumio a los objetos de política cargados

**Exportar desde el PCE, anonimizar, analizar
con Claude, revisar las propuestas y cargar
workloads no gestionados y listas de IP con
reconciliación por IP y trazabilidad completa
de cada corrida**

Eric Roscher – Illumio LATAM Pre-Sales Engineering

Septiembre 2026

Guía de trabajo preparada por Eric Roscher – Illumio LATAM Pre-Sales
Engineering — no es una publicación oficial de Illumio, Inc.

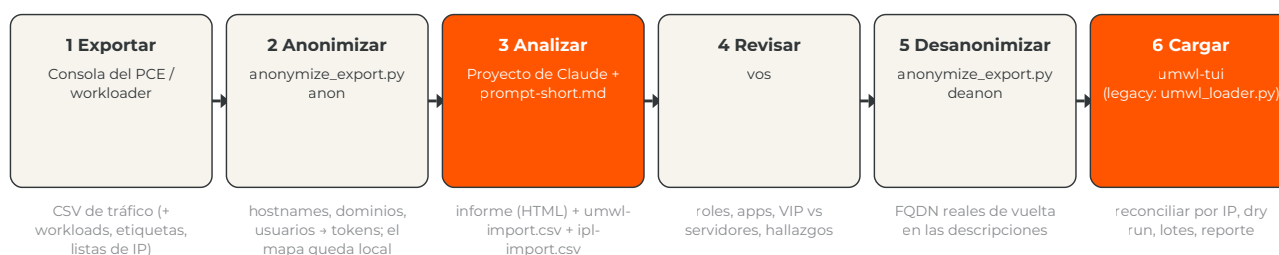
■ Tabla de contenidos

01	El flujo de trabajo de un vistazo	3
02	Requisitos y una carpeta de trabajo por cuenta de Illumio + PCE	5
03	Inicialización	8
04	Exportar desde el PCE	11
05	Anonimización	15
06	Correr el análisis con Claude	17
07	Objetos de política y convención de nombres	19
08	umwl-tui paso a paso	22
09	umwl-tui y workloader	26
10	Iterar: exports v2	29
11	Solución de problemas y lista de control	29
12	Apéndice A — Columnas del export	32
13	Apéndice B — Archivos del repositorio	33

■ El flujo de trabajo de un vistazo

Esta guía cubre el camino completo desde un export de tráfico del PCE hasta los objetos de política cargados en ese mismo PCE: exportar los datos correctos, seudonimizarlos, correr el análisis de flujos con Claude, revisar lo que vuelve, restaurar los nombres reales y cargar los workloads no gestionados y las listas de IP. Los seis pasos de abajo son el esqueleto del documento; cada uno tiene su propia sección.

LOS SEIS PASOS: EXPORTAR, ANONIMIZAR, ANALIZAR, REVISAR, DESANONIMIZAR, CARGAR



Nunca sale de la carpeta de trabajo: pce.yaml (API key), anon-map.json (el mapa de seudónimos), runs/ (inventarios y logs)

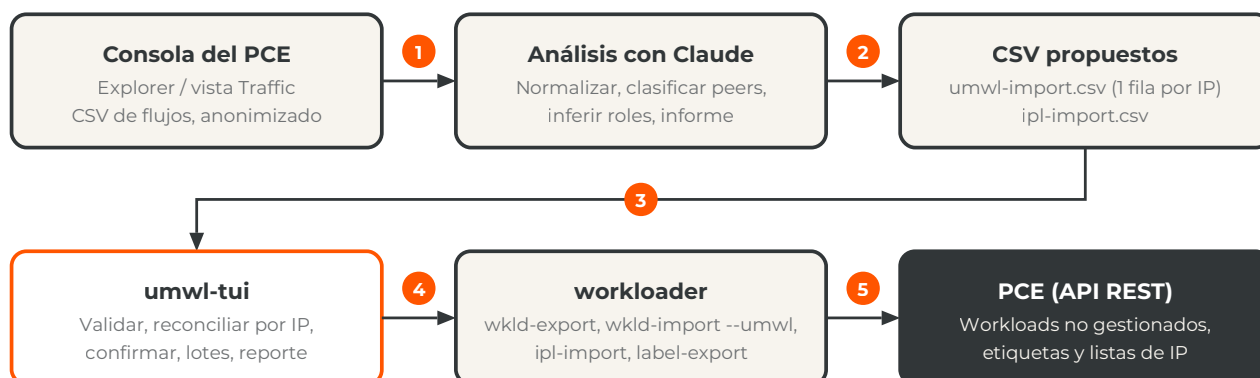
1. Export de tráfico de la vista Explore › Traffic (obligatorio) más los workloads, etiquetas, tipos de etiqueta y listas de IP que ya existen en el PCE (recomendado). Sección 4.
2. `anonymize_export.py anon` reemplaza hostnames, dominios, el nombre del cliente y los usuarios personales por tokens consistentes; el mapa queda en la carpeta de trabajo. Sección 5.
3. Un Proyecto de Claude con `docs/prompts/context.md` como instrucciones y el prompt corto por conversación devuelve el informe y los dos CSV. Sección 6.
4. Una persona revisa roles, aplicaciones, la distinción VIP frente a front-ends y los hallazgos; las correcciones vuelven a la misma conversación. Sección 6.
5. `anonymize_export.py deanon` devuelve los FQDN reales a las descripciones de los CSV propuestos. Sección 5.
6. `umwl-tui` reconcilia cada fila por IP contra el PCE, hace el dry run, pregunta, escribe por lotes y deja el reporte. Secciones 8 y 9.

Qué es el producto

`umwl-tui` es una aplicación de terminal de pantalla completa escrita en Go. Toma los workloads no gestionados (UMWL) y las listas de IP que propone el análisis, ya en formato CSV, y los carga en el PCE a través de `workloader`, la herramienta de línea de comandos de código abierto para la API REST del PCE publicada en GitHub como `brian1917/workloader`¹. Lo que la aplicación agrega sobre un import masivo es justamente lo que `workloader` no hace por sí solo: validar el CSV, reconciliar cada IP contra el inventario real del PCE, preguntarle a una persona antes de cada decisión que importa, hacer el dry run, escribir por lotes con reintento y salto, verificar y persistir todo lo que decidió u observó en `runs/`

`<timestamp>/` . Es el método principal que describe esta guía; la implementación anterior en Python de terminal plana, `legacy/umwl_loader.py` , se conserva en el repositorio solo como respaldo y sigue los mismos diez pasos.

FLUJO COMPLETO: DEL EXPORT DE TRÁFICO A LOS OBJETOS EN EL PCE



1. Export de la vista Explore › Traffic con la ventana y los filtros de la prueba de concepto, seudonimizado antes de salir de la carpeta de trabajo; se abre como texto, nunca se guarda desde Excel.
2. Normalización (IP mutiladas, fechas, truncamiento), criterio workload no gestionado frente a lista de IP, evidencia por puerto, proceso y volumen, modelo de etiquetas y hallazgos.
3. El análisis entrega los CSV en el contrato de `wkld-import` e `ipl-import` , con los valores de etiqueta propios del cliente y el comentario de evidencia en `description ; deanon` restaura los nombres reales antes de cargar.
4. `umwl-tui` llama a los subcomandos de workloader: primero solo lectura (inventario), después dry run y, tras confirmar, escritura por lotes.
5. workloader habla con la API REST del PCE usando el `pce.yaml` de la carpeta de trabajo.

UNA CARPETA DE TRABAJO POR CUENTA DE ILLUMIO + PCE

workloader lee `./pce.yaml` de la carpeta donde se ejecuta (salvo `--config-file` o `ILLUMIO_CONFIG`)² y `umwl-tui` escribe ahí mismo `runs/<timestamp>/` . La unidad de aislamiento es cada combinación **cuenta u organización de Illumio + PCE** (cada perfil distinto de `pce.yaml` : FQDN, puerto, org id, API key); compartir carpeta entre dos es arriesgarse a importar en el tenant equivocado. Estructura en la sección 2; comandos en la 3.

FUENTES — DOCUMENTACIÓN OFICIAL

1. workloader (brian1917) — README, releases v12.1.9 — github.com/brian1917/workloader
2. workloader — `cmd/root.go`: orden de búsqueda del archivo de configuración (`--config-file`, `ILLUMIO_CONFIG`, `./pce.yaml`) y flags persistentes — github.com/brian1917/workloader/blob/master/cmd/root.go

■ Requisitos y una carpeta de trabajo por cuenta de Illumio + PCE

`umwl-tui` es un único binario estático sin más dependencia en tiempo de ejecución que el binario de `workloader` en la carpeta de trabajo o en el PATH. Python 3.9 o superior solo hace falta para `anonymize_export.py` (biblioteca estándar, sin `pip install`) y para el loader heredado. Lo que sí necesita el flujo es una identidad con permiso de escritura en el PCE y acceso HTTPS a su API.

REQUISITOS Y ESTRUCTURA DE CARPETAS: UNA POR CUENTA + PCE

Mac (Apple Silicon o Intel)

umwl-tui

binario Go · release o make build

workloader v12.x

binario Intel + Rosetta, o go build

`~/illumio/cliente-a/scp57-org12/`

`umwl-tui`

`workloader`

`pce.yaml`

`kit/`

`cliente-a-umwl-import.csv`

`cliente-a-ipl-import.csv`

`anon-map.json`

`runs/20260903-101500/`

la app Go (release de GitHub)

binario; lo descarga umwl-tui

SOLO esta cuenta + PCE

clon: `anonymize_export.py`, ...

después de `deanon`

mapa de seudónimos; nunca sale

inventario, logs, reporte

`~/illumio/cliente-a/onprem-prod/`

`umwl-tui workloader pce.yaml ...`

misma cuenta, otro PCE

`~/illumio/cliente-b/<pce-o-org>/`

`umwl-tui workloader pce.yaml ...`

otra cuenta, misma estructura

1

PCE SaaS scp57

org 12 · HTTPS 443
API key con rol de escritura

2

PCE on-prem prod

misma cuenta, otro `pce.yaml`

3

PCE de cliente-b

otra cuenta, otra carpeta

1. Un PCE SaaS (un FQDN) puede alojar varias organizaciones; cada org id es un tenant distinto con su propia carpeta. `workloader` autentica con el `pce.yaml` de la carpeta actual; el preflight muestra el resultado de `pce-list` antes de seguir.
2. El mismo cliente con otro PCE (on-prem, DR, otra región) es otra carpeta con su propio `pce.yaml`; los binarios pueden compartirse con un symlink.
3. Otra cuenta: misma estructura, nunca el mismo `pce.yaml`, nunca el mismo `anon-map.json`.

La unidad de aislamiento: cuenta u organización de Illumio + PCE

Un FQDN de PCE SaaS puede alojar varias organizaciones (el `org id` es el número que aparece en la URL de la consola, `.../orgs/<id>/...`), y un mismo cliente puede tener varios PCE: SaaS y on-prem, producción y DR, regiones. Por eso la unidad de aislamiento no es "el cliente" ni "el PCE" sino cada combinación **cuenta u organización + PCE**, es decir, cada perfil distinto de `pce.yaml` (FQDN, puerto, org id y API key): el mismo cliente con dos orgs en el mismo PCE son dos carpetas.

workloader admite varios perfiles en un solo `pce.yaml`, con un `default_pce_name` y el flag `--pce <nombre>` para elegir uno por comando⁵. El kit no se apoya en eso a propósito: un `--pce` olvidado o un default equivocado cargaría los objetos en el tenant incorrecto. Nomenclatura sugerida: `~/illumio/<cuenta>/<pce-o-org>/`, por ejemplo `~/illumio/cliente-a/scp57-org12/`.

COMPONENTE	DETALLE
Sistema operativo	macOS en Apple Silicon o Intel (probado), o Linux. La terminal debe tener al menos 80×24; la interfaz está en inglés.
<code>umwl-tui</code>	Binarios precompilados adjuntos a los releases de GitHub del kit (<code>umwl-tui-darwin-arm64</code> , <code>umwl-tui-darwin-amd64</code> , <code>umwl-tui-linux-amd64</code> , <code>umwl-tui-linux-arm64</code> , más <code>SHA256SUMS</code>). En una Mac: <code>chmod +x</code> y <code>xattr -d com.apple.quarantine umwl-tui</code> . Alternativa: Go 1.24+ y <code>make build</code> desde un clon.
workloader v12.x	Preferentemente compilado desde el fuente por el preflight de <code>umwl-tui</code> (tecla <code>b</code>): clona <code>brian1917/workloader</code> ¹ en el tag del último release dentro de <code>workloader-src/</code> y ejecuta <code>go build</code> , lo que da un binario nativo para la máquina (arm64 en Apple Silicon) con versión reproducible. Requiere Go 1.24+ y <code>git</code> (<code>brew install go</code> ; <code>git</code> viene con <code>xcode-select --install</code>). El motivo para preferirlo: el release de macOS upstream (<code>mac-v12.1.9.zip</code>) es un binario solo Intel que corre bajo Rosetta 2. La descarga del release queda como alternativa (tecla <code>d</code>) para máquinas sin Go. Un binario que ya exista en cualquier lugar del disco se puede usar tal cual (tecla <code>w</code> , recordado en <code>umwl-tui.json</code>).
Carpeta de trabajo	Una por cuenta u organización + PCE: contiene <code>umwl-tui</code> , <code>workloader</code> , <code>pce.yaml</code> , los exports y los CSV, <code>anon-map.json</code> y <code>runs/</code> . Un clon del repositorio en <code>kit/</code> es opcional (aporta <code>anonymize_export.py</code> , los prompts y los CSV de ejemplo).
API key del PCE	Menú de usuario → My API Keys → Add ; guardar el <i>Authentication Username</i> (<code>api_...</code>) y el <i>Secret</i> , que se muestran una sola vez. Hereda el rol del usuario: para crear workloads, etiquetas y listas de IP hace falta Global Organization Owner o Global Administrator ^{2,4} . En SaaS el <code>org id</code> es el número de la URL.
Red	HTTPS desde la estación de trabajo al FQDN del PCE (443 en SaaS, normalmente 8443 on-prem). workloader no necesita nada más ³ . Descargar workloader desde el preflight necesita HTTPS a <code>github.com</code> .

PCE.YAML CONTIENE LA API KEY; ANON-MAP.JSON CONTIENE LOS NOMBRES REALES

No copies ninguno de los dos entre carpetas, ni los adjuntes a una conversación, ni los subas a un repositorio. El `.gitignore` del kit excluye ambos, junto con los binarios, los logs, `runs/` y las copias `*.anon.csv` / `*.real.csv`. workloader crea `pce.yaml`; restringilo con `chmod 600 pce.yaml`.

FUENTES — DOCUMENTACIÓN OFICIAL

1. workloader v12.1.9 — assets del release (mac-v12.1.9.zip, linux_amd64, linux_arm, windows) — github.com/brian1917/workloader/releases/tag/v12.1.9
2. Illumio Core 24.2 — REST API — API Keys (My API Keys, la key hereda el rol del usuario) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/REST-APIs/out/en/rest-apis/authentication-and-api-user-permissions/api-keys.html
3. workloader — README: "The only requirements for workloader are the ability to run the executable and connect to the PCE over HTTPS" — github.com/brian1917/workloader
4. Illumio Core 25.4 — Admin — Roles, Scopes, and Granted Access — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Admin/out/en/pce-administration/role-based-access-control/roles-scopes-and-granted-access.html
5. workloader — README y cmd/pcemgmt/addpce.go: varios PCE en un pce.yaml, default_pce_name y el flag --pce — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go

■ Inicialización

Estos pasos se hacen una vez por combinación cuenta + PCE. Dejan una carpeta con el binario de `umwl-tui`, el binario de workloader y un `pce.yaml` probado; después, cada carga es un solo comando. El repositorio del kit es privado: hace falta una sesión de GitHub (`gh auth login` o una clave SSH cargada en la cuenta) para descargar el release o clonar.

INICIALIZACIÓN: DE LA CARPETA VACÍA A LA PRIMERA CARGA



1. La carpeta es la unidad de aislamiento; todo lo que sigue se ejecuta con esa carpeta como directorio actual.
2. El binario sale del release de GitHub (checksum en `SHA256SUMS`) o de `make build` en un clon; el clon en `kit/` también aporta `anonymize_export.py` y los prompts.
3. `--setup-only` corre solo el preflight: ubica `workloader` o lo compila desde el fuente (`b` , nativo; `d` descarga el release como alternativa), abre el formulario de `pce-add --api-key` cuando `pce.yaml` no tiene ningún PCE y prueba la conexión con un `label-dimension-export` de solo lectura.
4. Antes de escribir nada, un export de solo lectura demuestra que la API key funciona y que el inventario es el del tenant esperado.
5. Los CSV que vuelven del análisis, revisados y con los nombres reales restaurados, van a la carpeta de trabajo con el nombre del cliente.
6. `umwl-tui` recorre los pasos 0 a 10 de la sección 8; escribe en el PCE solo después de la confirmación "Write to the PCE?" que sigue al dry run.

Comandos

TERMINAL · A. CARPETA DE TRABAJO PARA ESTA CUENTA + PCE

```
mkdir -p ~/illumio/cliente-a/scp57-org12 && cd ~/illumio/cliente-a/scp57-org12
```


TERMINAL · B. UMWL-TUI DESDE EL RELEASE DE GITHUB (REPOSITORIO PRIVADO: ANTES GH AUTH LOGIN)

```
gh release download --repo roschereric/illumio-workloader-import-kit \
  --pattern 'umwl-tui-darwin-arm64' --pattern SHA256SUMS
shasum -a 256 -c SHA256SUMS --ignore-missing
mv umwl-tui-darwin-arm64 umwl-tui && chmod +x umwl-tui && xattr -d com.apple.quarantine umwl-tui
# Mac Intel: umwl-tui-darwin-amd64 · Linux: umwl-tui-linux-amd64 / umwl-tui-linux-arm64

# alternativa: compilarlo (Go 1.24+); el clon también te da anonymize_export.py y docs/prompts/
git clone https://github.com/roschereric/illumio-workloader-import-kit.git kit
(cd kit && make build) && cp kit/umwl-tui .
```

TERMINAL · C. WORKLOADER Y PCE.YAML CON EL PREFLIGHT DE UMWL-TUI

```
./umwl-tui --setup-only
```

El preflight muestra tres comprobaciones: *workloader binary*, *pce.yaml* y *PCE connection*, y un bloque *Environment* con la carpeta, el SO/CPU y el toolchain encontrado (`go` , `git` , `brew`). Busca *workloader* en este orden: `--workloader <ruta>` , la ruta guardada en `./umwl-tui.json` o `~/.config/umwl-tui/config.json` , `./workloader` , el PATH. La fila del chequeo imprime la versión y la arquitectura de CPU de lo que encontró — `arm64` , `native` , o una advertencia cuando un binario Intel correría bajo Rosetta 2 en un Mac Apple Silicon. Tres teclas instalan o eligen el binario:

TECLA	VÍA	QUÉ PASA
b	Compilar desde el fuente (recomendado)	Modal de confirmación y después <code>git clone --depth 1 --branch <último tag> https://github.com/brian1917/workloader</code> <code>workloader-src</code> y <code>CGO_ENABLED=0 go build -trimpath -ldflags "... utils.Version=\$(cat version) ..." -o ./workloader</code> . — los mismos flags que el workflow de release de <i>workloader</i> , así <code>workloader version</code> informa el tag. El clon recibe un nombre distinto al del binario (<code>workloader-src</code>) para que <code>./workloader</code> pueda convivir al lado. Si falta Go y hay Homebrew, el modal ofrece <code>brew install go</code> y después compila; sin Homebrew muestra los comandos manuales (bloque siguiente). La salida se ve en el panel de log; la primera compilación descarga los módulos Go de <i>workloader</i> (uno a tres minutos), las siguientes tardan segundos.
d	Descargar el release (alternativa)	Modal de confirmación y después el release precompilado para el SO actual (<code>curl -fsSL</code> desde github.com, <code>unzip</code> , <code>chmod</code> , en macOS <code>xattr -d com.apple.quarantine</code>). En darwin/arm64 el modal avisa que ese binario es solo Intel y correrá bajo Rosetta 2.
w	Usar un workloader en otro lugar	Abre el selector de archivos (escribí una ruta arriba con <code>tab</code> ; <code>~</code> funciona), verifica el bit de ejecución y lee la versión, y después pregunta dónde recordar la elección: <code>l</code> esta carpeta (<code>./umwl-tui.json</code>) o <code>u</code> este usuario, todas las carpetas (<code>~/.config/umwl-tui/config.json</code>). Ambos archivos guardan solo la ruta, sin secretos; el local tiene prioridad; <code>--workloader</code> en la línea de comandos los pisa para una ejecución. Así un binario compilado una vez en <code>~/tools/</code> sirve a todas las carpetas de trabajo sin tocar el PATH.

Después corre `pce-list`; cuando `pce.yaml` no existe o workloader responde "no pce configured", la tecla `p` abre un formulario con nombre corto, FQDN, puerto, API user, API secret (enmascarado), org id y el interruptor de verificación TLS, y ejecuta `pce-add --api-key ...` con esos valores². El secreto se pasa a workloader como argumento, nunca a través de un shell, y queda enmascarado en el panel de log y en el reporte. La tecla `t` repite la prueba de conexión (un `label-dimension-export` de solo lectura) y `r` vuelve a correr las tres comprobaciones. Con las tres en verde la pantalla dice "Setup complete."; `q` sale.

L · C'. LA MISMA COMPILACIÓN A MANO (CUANDO B NO PUEDE CORRER, O PARA COMPILAR UNA VEZ EN UNA CARPETA COMPARTIDA)

```
brew install go # una vez por Mac (Go 1.24+); git: xcode-select --install
git clone --depth 1 --branch v12.1.9 https://github.com/brian1917/workloader workloader-src
cd workloader-src
CGO_ENABLED=0 go build -trimpath -o ../workloader \
  -ldflags "-s -w -X github.com/brian1917/workloader/utils.Version=$(cat version) \
    -X github.com/brian1917/workloader/utils.Commit=$(git rev-list -1 HEAD)" .
cd .. && ./workloader version && file ./workloader # el tag, y "Mach-O 64-bit executable arm64"
# tags: git ls-remote --tags https://github.com/brian1917/workloader | tail
# actualizar después: git -C workloader-src fetch --tags && git -C workloader-src checkout v12.2.0, compilar de nuevo
# el resultado es un único archivo estático: cp workloader ~/tools/workloader, y después w en cada carpeta de trabajo
```

TERMINAL · D. PRUEBA DE CORDURA (SOLO LECTURA)

```
./workloader pce-list
./workloader wkld-export --output-file sanity.csv
# mirar los hostnames de sanity.csv: ¿es el inventario del tenant esperado?
```

ESTRUCTURA RESULTANTE

```
~/illumio/cliente-a/scp57-org12/
umwl-tui # la aplicación Go (binario del release o make build)
workloader # binario nativo compilado por el preflight (b) — o una ruta guardada en umwl-tui.json
workloader-src/ # clon de brian1917/workloader usado para compilar (gitignored)
umwl-tui.json # opcional: ruta a un binario workloader en otro lugar (sin secretos)
pce.yaml # SOLO esta cuenta + PCE; lo crea pce-add; nunca copiarlo
kit/ # clon opcional: anonymize_export.py, docs/prompts/, examples/
TrafficData 9_2_2026.csv # exports del PCE (sección 4)
anon-map.json # mapa de seudónimos (sección 5); nunca sale de esta carpeta
cliente-a-umwl-import.real.csv
cliente-a-ipl-import.csv
runs/ # una subcarpeta por corrida
```

SITUACIÓN	QUÉ HACER
e. Otra cuenta u otro PCE	Repetir a–d en una carpeta nueva. Nunca copiar <code>pce.yaml</code> ni <code>anon-map.json</code> . Los binarios pueden compartirse: <code>ln -s ~/illumio/bin/umwl-tui umwl-tui</code> para la aplicación y, para workloader, un symlink o la tecla <code>W</code> con alcance de usuario (<code>~/.config/umwl-tui/config.json</code>), que cada carpeta nueva toma automáticamente.
f. Actualizar las herramientas	Descargar el release nuevo (o <code>git -C kit pull && make -C kit build</code>). <code>umwl-tui --version</code> imprime la versión. Para workloader, <code>b</code> de nuevo: el clon existente se actualiza al tag más nuevo y se recompila.
Varios PCE en un mismo <code>pce.yaml</code>	No es el esquema del kit. Si igual existe, pasar siempre <code>--pce <nombre></code> a <code>umwl-tui</code> ; el preflight muestra el resumen de <code>pce-list</code> y la carpeta de trabajo antes de seguir.
Respaldo heredado	<code>python3 kit/legacy/umwl_loader.py --setup-only</code> y <code>python3 kit/legacy/umwl_loader.py <csv> --ipl ... --priority 1</code> siguen los mismos pasos en una terminal plana.

FUENTES — DOCUMENTACIÓN OFICIAL

1. workloader — `cmd/root.go`: el archivo de configuración es `--config-file`, si no `ILLUMIO_CONFIG`, si no `./pce.yaml` del directorio actual — github.com/brian1917/workloader/blob/master/cmd/root.go
2. workloader — `cmd/pcemgmt/addpce.go`: `pce-add` con `--api-key` toma `--api-user`, `--api-secret` y `--org`; sin él, pide correo y contraseña — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
3. GitHub CLI — `gh auth login`, `gh release download`, `gh repo clone` — docs.github.com/en/github-cli/github-cli/about-github-cli

■ Exportar desde el PCE

Tres clases de datos alimentan el análisis. El export de tráfico es obligatorio; los exports de workloads, etiquetas y listas de IP hacen que las propuestas reutilicen la nomenclatura existente del cliente y eviten duplicados. Las imágenes de la consola en esta sección son **esquemas**: muestran dónde están los controles y qué esperar; los rótulos exactos varían un poco entre versiones de la consola (los pasos siguen la documentación oficial de Illumio Core 23.5–25.x). En consolas 22.x–23.x la vista se llama **Explorer**; en 24.x/25.x los mismos datos están en la vista **Traffic** bajo **Explore**¹.

4.1 Export de tráfico (Explore › Traffic) — obligatorio

CONSOLA DEL PCE — EXPLORE › TRAFFIC: FILTROS, VENTANA, RESULTS SETTINGS, EXPORT

PCE console — Explore › Traffic esquema, no captura; los rótulos varían según la versión

Dashboard
Explore
 Map
Traffic
 Mesh
 App Groups
Servers & Endpoints
Policy Objects
 Labels
 IP Lists
 Services
Policies
Settings

2 Source: any → Destination: label Ordering · Service: any 4 Run

3 Time: Last 7 days ▼ (custom) 5 Reported ▼ | Draft More ▼ › Results Settings Load Results

Source	Destination	Port/Proto	Process	Flows	Policy decision	First / Last
10.20.1.11	ord-app-01...	8080 TCP	java	21,025	Unenforced Deny	Sep 1 → Sep 2
10.10.4.21	ord-app-01...	10050 TCP	zabbix_agentd	24,751	Unenforced Deny	Sep 1 → Sep 2
10.30.1.21	bil-app-01...	7001 TCP	java	170,965	No Rule	Aug 26 → Sep 2
bil-app-01	169.254.169.254	53 UDP	—	5,225,842	Allowed	Aug 26 → Sep 2
...	...	...	...	...	...	...

Showing 5,000 of 5,000 results — justo el límite: subilo en More › Results Settings (paso 5)

6 ↓ Export

- Explore › Traffic** (Explorador en consolas anteriores). Esperado: la barra de filtros (Source / Destination / Service), el selector de tiempo y una tabla de resultados vacía.
- Filtros.** Poné los workloads del alcance como Destination y otra vez como Source en una segunda consulta, o filtrá por sus etiquetas (por ejemplo la etiqueta Application del grupo de la POC) y dejá el otro lado abierto. No filtres por puerto. Agregá exclusiones (**More › Show Exclusion Filters**) para los escáneres de vulnerabilidades conocidos una vez identificados: un solo escaneo puede consumir todo el presupuesto de resultados.
- Ventana de tiempo.** Siete días es la primera pasada correcta (un ciclo semanal completo: backups, procesos batch, parcheo). Ventanas más largas llegan antes al tope de resultados; corré una segunda consulta más larga para los flujos poco frecuentes si hace falta.
- Run.** Esperá a que la consulta termine. Esperado: la tabla se llena y el pie muestra "N of M results". Las consultas de las últimas 24 horas reaparecen en **Load Results** sin volver a ejecutarse¹.
- More › Results Settings.** Subí el máximo de resultados antes de exportar. La consola muestra hasta 100.000 filas y el export puede llegar a 200.000^{1, 5}; el default de una consulta guardada puede ser de apenas 5.000. Si el export tiene exactamente la cantidad de filas del tope, está truncado: el análisis dirá qué lo consumió y qué filtro agregar.
- Export.** El botón descarga un CSV (`TrafficData <fecha>.csv`) con una columna por tipo de etiqueta²; la cabecera completa está en el Apéndice A.

Comprobaciones sobre el archivo antes que nada. Abrilo como texto (cualquier editor, `head`, Python). Nunca lo abras en Excel y lo guardes: Excel le quita los puntos a las IP que parecen números (`10.20.2.20` se convierte en `1020220`) y reescribe las fechas. Si alguien ya lo hizo, conservá el archivo igual: el análisis reconstruye las IP de forma determinista, pero decilo en el prompt. Contá las filas contra el tope del paso 5. Anotá el formato de fecha (el export mezcla `MM/DD/YYYY HH:MM` y, en algunas versiones, `YYYY-DD-MM` ; el análisis maneja ambos, pero indicá la ventana que esperás). La columna `Reported by` dice qué lado tenía el VEN; las columnas `Enforcement` dicen qué IP son workloads. La columna de decisión de política tiene la semántica Allowed, Blocked y Potentially Blocked³.

TERMINAL · ALTERNATIVAS AL EXPORT DE LA CONSOLA

```
# workloader: "Export traffic data"; fechas en GMT, --max-results por defecto 100000, máximo 200000
./workloader traffic --start 2026-08-17 --end 2026-09-01 --max-results 200000 --output-file TrafficData.csv
# API REST (la misma consulta asíncrona que corre la consola; max_results hasta 200.000):
# POST /api/v2/orgs/<org>/traffic_flows/async_queries → GET .../async_queries/<uuid> → download
```

Las columnas del CSV de `workloader traffic` no son idénticas a las del export de la consola; las instrucciones del análisis normalizan los nombres de columna, así que sirve cualquiera de los dos.

4.2 Workloads, etiquetas y tipos de etiqueta — recomendado

CONSOLA DEL PCE — SERVERS & ENDPOINTS › WORKLOADS, Y EL EQUIVALENTE EN LA TERMINAL

PCE console — Servers & Endpoints › Workloads esquema, no captura; los rótulos varían según la versión

Dashboard
Explore
Servers & Endpoints ¹
Workloads
VENs
Policy Objects
Policies
Settings

Filter: Enforcement · Labels · Managed / Unmanaged ... Add ▼ ² ↓ Export

Name	Hostname	Interfaces	Enforcement Role	App	Env	Loc
ord-app-01	ord-app-01.lab...	eth0:10.20.2.20	Visibility Only	App	Ordering	Production DC1
Front-End Ord. 10.20.1.11	—	eth0:10.20.1.11	(unmanaged)	Web	Ordering	Production DC1
DNS Corporativo 10.10.0.53	—	eth0:10.10.0.53	(unmanaged)	DNS	Shared Services	Production DC1

Equivalente en la terminal (recomendado, sin clics):

```
./workloader wkld-export --output-file pce-workloads.csv
./workloader label-export --output-file pce-labels.csv
./workloader label-dimension-export --output-file pce-label-types.csv
```

- Servers & Endpoints › Workloads** lista los workloads gestionados (VEN) y no gestionados con sus etiquetas; el filtro acota por enforcement, etiquetas y gestionado/no gestionado.
- Export** descarga la lista como CSV: los mismos datos que `workloader wkld-export`, que es lo que usa `umwl-tui` en el paso 2. Archivo esperado: una fila por workload con `href`, `hostname`, `name`, `interfaces`, `managed` y las etiquetas⁴; las propuestas se reconcilian contra él por IP.

Desde la carpeta de trabajo, una vez que existe `pce.yaml` (`./umwl-tui --setup-only` lo crea), los tres comandos de terminal de la imagen producen `pce-workloads.csv` (una fila por workload), `pce-labels.csv` (`href, key, value` : los valores existentes por tipo) y `pce-label-types.csv` (una fila por tipo de etiqueta). Equivalentes en la consola: **Servers & Endpoints › Workloads › Export**, **Policy Objects › Labels** (filtrar por tipo, Export) y **Settings › Label Settings** para los tipos. Una columna del CSV de propuestas que no sea un tipo acá es ignorada en silencio por workloader; creá el tipo primero (`workloader label-dimension-import`, sección 11).

4.3 Etiquetas, tipos de etiqueta y listas de IP — opcional pero útil

CONSOLA DEL PCE — POLICY OBJECTS › LABELS, SETTINGS › LABEL SETTINGS, POLICY OBJECTS › IP LISTS

PCE console — Policy Objects › Labels › IP Lists › Settings › Label Settings esquema, no captura; los rótulos varían según la versión

Dashboard

Explore

Servers & Endpoints

Policy Objects

Labels

Label Groups

IP Lists

Services

Settings

Label Settings

Labels — filter by type: role · app · env · loc

↓ Export

role	Web, App, Database, LoadBalancer, DNS ...
app	Ordering, Billing, Shared Services ...
env	Production, Staging
loc	DCI, Cloud

Equivalente en la terminal:

```
./workloader label-export --output-file pce-labels.csv
./workloader label-dimension-export --output-file pce-label-types.csv
./workloader ipl-export --output-file pce-iplists.csv
```

- 1. Policy Objects › Labels:** los valores existentes por tipo. Exportalos para que el análisis reutilice los valores y el estilo de nombres del cliente.
- 2. Settings › Label Settings:** los tipos de etiqueta. Una columna del CSV que no sea un tipo es ignorada por workloader; creá el tipo primero.
- 3. Policy Objects › IP Lists:** las listas existentes (redes corporativas, DNS, NTP). `./workloader ipl-export --output-file pce-iplists.csv` o **Policy Objects › IP Lists › Export**. Adjuntalo para que las reglas propuestas referencien las listas existentes en lugar de crear duplicados.

Capturas de la nomenclatura. Dos o tres capturas de workloads no gestionados y etiquetas existentes en la consola (patrón de nombres, prefijos de etiqueta, formato de interfaz) le permiten al asistente copiar las convenciones del cliente con exactitud. El kit no impone una convención de etiquetas propia (valores simples como recomienda Illumio, o el estilo que ya use el PCE); sus valores por defecto son nombre del workload `"<rol> <descriptivo> <IP>"`, hostname vacío e interfaz `eth0:<ip>` (sección 7).

FUENTES — DOCUMENTACIÓN OFICIAL

- Illumio Core 23.5 — Visualization — About the Visualization Tools (Explore › Traffic, filtros, Results Settings, límites 100.000 UI / 200.000 export, Load Results) — product-docs-repo.illumio.com/Tech-Docs/Core/23.5/Visualization/out/en/visualization-visualization-tools/about-the-visualization-tools.html
- Illumio Core 24.2 — Visualization — Traffic Table (Export to CSV, una columna por tipo de etiqueta) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Visualization/out/en/visualization-visualization-tools/traffic-table.html
- Illumio Core 23.2 — Visualization — Traffic Table (semántica Allowed / Blocked / Potentially Blocked) — product-docs-repo.illumio.com/Tech-Docs/Core/23.2/Visualization/out/en/visualization-user-guide-23-2/visualization-visualization-tools/traffic-table.html
- workloader — [cmd/wkldexport/headers.go](https://github.com/brian1917/workloader/blob/master/cmd/wkldexport/headers.go): columnas del export (hostname, name, interfaces, public_ip, href, managed, ven_href...); README: traffic, label-export, label-dimension-export, ipl-export — github.com/brian1917/workloader/blob/master/cmd/wkldexport/headers.go
- Illumio Core 22.5 — REST API — Explorer: traffic_flows/async_queries, max_results 200.000 — product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html

■ Anonimización

Los exports salen de la carpeta de trabajo para ser analizados. Antes de que salgan, todo lo que identifica al cliente se reemplaza por tokens consistentes, y solo queda intacta la evidencia que el análisis necesita. Las IP privadas son la clave que une el export, las propuestas y el inventario del PCE: nunca se anonimizan.

Qué sale de la carpeta y qué no debe salir

EL ASISTENTE NECESITA	EL ASISTENTE NO NECESITA
IP privadas (exactas), puertos, protocolos, nombres de proceso, usuarios de cuentas de servicio (<code>root</code> , <code>oracle</code> , <code>zabbix</code> ... son evidencia de rol), conteos de flujos, fechas, valores de etiqueta y la forma de los hostnames (qué host es cuál, qué capa sugiere un dominio).	El nombre del cliente, los hostnames y FQDN reales, los usuarios personales, las IP públicas del cliente ni nada de <code>pce.yaml</code> .

NUNCA SUBIR

`pce.yaml` (API key), `anon-map.json` (el mapa de seudónimos), `runs/` (inventarios con nombres reales), API keys o tokens de sesión pegados desde una terminal.

La herramienta: `anonymize_export.py`

Seudonimización consistente y reversible, solo biblioteca estándar de Python. La misma entrada siempre produce el mismo token, en todas las columnas y en todas las corridas que comparten el archivo de mapa, así que el export de tráfico y el export de workloads pueden anonimizarse por separado y seguir coincidiendo.

TERMINAL · ANTES DEL ANÁLISIS (CARPETA DE TRABAJO; EL MAPA SE CREA O ACTUALIZA EN `ANON-MAP.JSON`)

```
python3 kit/anonymize_export.py anon "TrafficData 9_2_2026.csv" -o traffic.anon.csv --map anon-map.json \
  --customer "Acme S.A." --customer acme --domain acme.com --domain acme.local --public-ips
python3 kit/anonymize_export.py anon pce-workloads.csv -o pce-workloads.anon.csv \
  --map anon-map.json --domain acme.com
```

después del análisis: restaurar los nombres reales en los CSV propuestos antes de cargar

```
python3 kit/anonymize_export.py deanon G1-umwl-import.csv -o G1-umwl-import.real.csv --map anon-map.json
```

QUÉ REEMPLAZA	CON QUÉ LO REEMPLAZA
Hostnames y FQDN en las columnas Name, Hostname y FQDN	Formato <code>host-0001.company.com</code> , conservando la profundidad del dominio (<code>unix.corp.acme.com</code> pasa a <code>unix.dept.company.com</code>) para que las capas sigan siendo reconocibles.
El nombre del cliente (<code>--customer</code> , repetible, palabras completas sin distinguir mayúsculas)	<code>Cliente</code> , en todos lados.
Los dominios dados con <code>--domain</code> y sus subdominios	<code>company.com</code> , <code>dept.company.com</code> . Sin <code>--domain</code> , se seudonimiza cada dominio visto.
Usuarios personales (Consuming Username, Username)	<code>user-01</code> ...; las cuentas de servicio conocidas se conservan porque cargan la evidencia de rol.
Las IP públicas del cliente (solo con <code>--public-ips</code>)	Rangos de prueba <code>203.0.113.x</code> / <code>198.51.100.x</code> . Las direcciones RFC 1918, CGNAT y link-local quedan intactas.
Sin tocar	IP privadas, puertos, protocolos, procesos, valores de etiqueta, conteos de flujos, fechas.

PASÁ SIEMPRE --DOMAIN CON LOS DOMINIOS DEL CLIENTE

Los FQDN de terceros (`*.trendmicro.com` , `*.oraclecloud.com` , `*.googleusercontent.com`) quedan entonces visibles, y son evidencia: consolas de EDR, metadata de nube, SaaS. El archivo de mapa se escribe con modo 600 y está en `.gitignore` .

Comprobación de fugas. La herramienta termina revisando la salida en busca de dominios reales, el nombre del cliente o hostnames cortos reales que hayan quedado, e imprime una advertencia si encuentra alguno. Lé la advertencia: si un hostname también se usa como valor de etiqueta o dentro de una descripción, agrégalo con `--customer` para que se reemplace en todos lados.

Qué cambia la anonimización en el análisis

Los roles deben salir de puertos, procesos y usuarios, nunca de los nombres: el asistente recibe esa instrucción en `context.md` , y por eso también los nombres de los workloads en las propuestas son "rol + IP". La `description` de cada workload propuesto lleva el FQDN seudonimizado ("FQDN en export (puede estar anonimizado): `host-0012.dept.company.com`"); `deanon` lo restaura antes de cargar, así el objeto en el PCE muestra el nombre real en su descripción mientras el informe compartido con terceros no. Los valores de etiqueta no se anonimizan: son la nomenclatura del cliente y deben coincidir con el PCE. Si un valor de etiqueta contiene el nombre del cliente, renómbrales primero en el PCE o agrégalo a `--customer` .

FUENTES — REPOSITORIO

1. `illumio-workloader-import-kit` — `anonymize_export.py` (`anon` / `deanon`, `--customer`, `--domain`, `--public-ips`, comprobación de fugas) y `.gitignore` — github.com/roschereric/illumio-workloader-import-kit

■ Correr el análisis con Claude

Un Proyecto de Claude conserva instrucciones y archivos de referencia entre conversaciones, que es exactamente lo que este flujo necesita: cada export nuevo es una conversación nueva, pero las reglas de parseo, los contratos de los CSV y la nomenclatura no deben explicarse otra vez cada vez. El método vive en dos archivos del repositorio: `docs/prompts/context.md` (el método completo: entradas, normalización, el criterio workload frente a lista de IP incluida la regla del balanceador, el modelo de etiquetas, el formato de hallazgos, los entregables y sus contratos CSV, el procedimiento v2 y la puerta de calidad) y `docs/prompts/prompt-short.md` (el mensaje por conversación, variantes v1 y v2).

Configuración del Proyecto (una vez)

1

Crear el proyecto

claude.ai › Projects › New project. Nombralo por la práctica, no por un cliente (por ejemplo `Illumio - flow analysis & object proposals`): un proyecto, una conversación por cliente y export. Si dos clientes usan nomenclaturas distintas, indicá la convención en el prompt corto (el default) o creá un proyecto por cliente.

2

Instrucciones del proyecto

Pegá el texto completo de `docs/prompts/context.md`. Cuando cambien las convenciones (un tipo de etiqueta nuevo, otro patrón de nombres, un sitio nuevo), editá las instrucciones una vez en lugar de repetirlo en las conversaciones.

3

Conocimiento del proyecto (archivos)

Agregá `context.md` otra vez como archivo (para que pueda citarse), el par ficticio de ejemplo `examples/sample-umwl-import-v2.csv` y `examples/sample-ipl-import-v2.csv` como referencias de formato, y la línea de cabecera de un export de tráfico (`docs/export-columns.txt`). No agregues exports reales de clientes como conocimiento permanente: adjuntalos por conversación.

4

Lo que nunca va al proyecto

`pce.yaml`, API keys, `anon-map.json`, ni el nombre del cliente en el título o la descripción del proyecto.

Por conversación

Adjuntá en un solo mensaje: el export de tráfico anonimizado, el export de workloads anonimizado, los exports de etiquetas y tipos de etiqueta, el export opcional de listas de IP, las capturas opcionales de la consola y el texto de `prompt-short.md` con las líneas de id de grupo, sitios y alcance editadas. Nombrá la conversación `<código de cliente> - <fecha del export> - v1|v2`. Sin un Proyecto (por ejemplo desde la cuenta de otra persona), adjuntá `context.md` en cada pedido y empezá el mensaje con "Follow context.md"; los resultados son los mismos, solo cambia la comodidad. Pedí las correcciones en la misma conversación ("10.30.1.x son servidores front-end, no el LB; el LB es la VIP"): el asistente regenera el CSV y el informe. Después corré `deanon` sobre los CSV (sección 5).

Qué vuelve y qué revisar antes de aceptarlo

SALIDA	REVISAR
Informe (un solo archivo HTML, en español, sin nombre del cliente)	Los números del resumen ejecutivo coinciden con las tablas; el veredicto de truncamiento; el diagrama de capas coincide con lo que sabés de la arquitectura.
<code><grupo>-umwl-import.csv</code>	Rol <code>LoadBalancer</code> solo en VIPs; los front-ends detrás de una VIP son <code>Web</code> ; los valores de etiqueta reutilizan los del PCE (misma grafía, mismo estilo, sin prefijos inventados); nada inferido de hostnames; una fila por IP; nombres únicos; interfaces <code>eth0:<ip></code> ; <code>review</code> = PENDING.
<code><grupo>-ipl-import.csv</code>	Los CIDR coinciden con el plan de direccionamiento; ningún duplicado de una lista de IP existente.
Hallazgos S-xx	Cada uno tiene evidencia con conteos y fechas; se distinguen escaneos de artefactos de dirección; las afirmaciones de fin de soporte citan al fabricante.
Resumen en el chat	Ventana, truncamiento, hosts con VEN, cuántos workloads y listas propone, los tres hallazgos más importantes y qué necesita que confirmes.

LOS ENTREGABLES ESTÁN EN ESPAÑOL

El informe y las descripciones de los CSV que piden los prompts son customer-facing y están en español, porque así se usa este kit con clientes de LATAM. Para cambiar el idioma, editá las secciones "Inputs" y "Deliverables" de `context.md`.

FUENTES — REPOSITORIO

1. [illumio-workloader-import-kit — docs/prompts/context.md](#) (instrucciones del proyecto), [docs/prompts/prompt-short.md](#) (mensaje por conversación), [docs/export-columns.txt](#), [examples/*-v2.csv](#) — [github.com/roschereric/illumio-workloader-import-kit](#)

■ Objetos de política y convención de nombres

`umwl-tui` crea dos tipos de objeto y, como efecto secundario, valores de etiqueta. Saber qué es cada uno explica las decisiones del análisis y las de la aplicación.

OBJETO	QUÉ ES Y CUÁNDO SE USA
Workload no gestionado (UMWL)	Entidad de red sin VEN que se da de alta en el PCE para escribir reglas sobre ella; la política se aplica con las reglas del lado que tiene VEN ¹ . Para servidores concretos con rol identificable (DNS, NTP, Zabbix, Splunk, NetBackup, bases de datos, balanceadores, bastiones, escáneres). Uno por IP, con etiquetas.
Lista de IP	Colección estática de direcciones, rangos y FQDN ² . Para peers amplios o que no son servidores : VLAN de usuarios, subredes completas, metadata de nube (169.254.169.254), NTP público, consolas SaaS. También como atajo para escribir hoy una regla que migra a etiquetas después.
Etiquetas y tipos	Cuatro tipos por defecto (Role, Application, Environment, Location) más los que se creen en el PCE, hasta 20, cada uno con una clave única ³ . Los valores son texto libre; la guía de Illumio es mantenerlos simples y reflejar los nombres que la organización ya usa (<code>Web</code> , <code>Database</code> , <code>Production</code> , <code>AWS</code>): no hay un esquema obligatorio de prefijos ni separadores ⁴ . workloader crea valores nuevos, no tipos : una columna que no sea un tipo existente se ignora en silencio (el paso 3 de <code>umwl-tui</code> lo detecta).
Servicios	Objetos de puerto y protocolo. El informe los propone para las reglas; el kit no los crea (se cargan con <code>svc-import</code> o desde la consola).

POR QUÉ LA PRIORIDAD VA EN LA DESCRIPCIÓN

Es el único campo libre que viaja con el objeto al PCE y que se puede leer después en la consola.

`umwl-tui` la usa para filtrar (`--priority 1`) sin necesitar columnas extra que workloader ignoraría.

Convención de nombres

CAMPO	REGLA	EJEMPLO
Etiquetas	Una columna del CSV por <i>clave</i> de tipo de etiqueta (<code>role</code> , <code>app</code> , <code>env</code> , <code>loc</code> , claves personalizadas); el valor es el valor de etiqueta tal cual existe en el PCE (<code>label-export</code>), o uno nuevo en el mismo estilo. El kit no impone una convención propia: si el cliente prefija o escribe los valores de cierta manera, seguila; si el PCE está vacío, usá valores simples como recomienda Illumio. Role = función del host (<code>Web</code> , <code>App</code> , <code>Database</code> , <code>LoadBalancer</code> solo en VIPs — los front-ends detrás de una VIP son <code>Web</code> —, <code>DNS</code> , <code>Monitoring</code> , <code>Backup</code> ...); Application = la aplicación de negocio dueña del host, con valores propios para la infraestructura compartida (<code>Shared Services</code> , <code>Monitoring</code> , <code>Security Tools</code> , <code>Admin Access</code>); Environment <code>Production</code> / <code>Staging</code> ; Location = sitio o región como lo nombra el cliente	<code>DNS · Shared Services · Production · DC1</code>
<code>name</code>	Rol descriptivo seguido de la IP; es lo que se ve en la consola y lo que hace única a la fila	<code>Zabbix Server 10.10.4.21</code>
<code>hostname</code>	Vacío salvo que se conozca el hostname real. Los FQDN de los exports pueden estar seudonimizados; un hostname inventado confunde	(en blanco)
<code>interfaces</code>	<code>eth0:<ip></code> ; varias interfaces separadas por ;	<code>eth0:10.10.4.21</code>
<code>description</code>	Prefijo [<grupo> P<prioridad> conf:<Alta Media Baja>] seguido del comentario con la evidencia y el FQDN visto en el export	<code>[G1 P1 conf:Alta] Servidor Zabbix: consulta 10050/TCP...</code>
Listas de IP, servicios, rulesets	Nombres descriptivos en el estilo del cliente; la descripción de la lista de IP empieza con [<grupo>] y termina con el uso previsto en reglas	<code>Corporate DNS · Ordering Web 8080 · Ringfence Ordering</code>

review

Columna de trabajo: PENDING en v1;
PENDING / UPDATED / UNCHANGED en v2.
workloader la ignora

PENDING

FUENTES — DOCUMENTACIÓN OFICIAL

1. Illumio Core 24.5 — Security Policy — Workload Setup Using PCE Web Console (Add Unmanaged Workload) — product-docs-repo.illumio.com/Tech-Docs/Core/24.5/Security-Policy/out/en/security-policy-guide-24-5/workloads/workload-setup-using-pce-web-console.html
2. Illumio Core 24.2 — Getting Started — Policy Objects — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Getting-Started/out/en/policy-overview/policy-objects.html
3. Illumio Core 25.4 — Security Policy — Create a Label Type — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Security-Policy/out/en/security-policy-objects/about-labels-and-label-groups/label-types/create-a-label-type.html
4. Illumio Core 24.2 — Getting Started — Labeling Workloads lesson (valores de ejemplo Web, Database, production, staging, AWS; imitar los nombres de la infraestructura) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Getting+Started/out/en/application-ringfencing/labeling-workloads-lesson.html

■ umwl-tui paso a paso

Dos comandos cubren el uso normal, siempre desde la carpeta de trabajo (sección 3). El primero se corre una vez por carpeta; el segundo, una vez por CSV, después de `deanon`.

TERMINAL · CARPETA DE TRABAJO DE LA CUENTA + PCE

```
# 1. una vez por cuenta + PCE: workloader, pce.yaml (pce-add --api-key) y la prueba de conexión
./umwl-tui --setup-only

# 2. cargar los workloads de prioridad 1 y, al final, las listas de IP
./umwl-tui cliente-a-umwl-import.real.csv --ipl cliente-a-ipl-import.csv --priority 1

# otros flags: --pce NOMBRE · --workloader RUTA (por defecto ./workloader, después el PATH)
# --chunk 20 (filas por llamada a wkld-import) · --runs ./runs · --priority 1,2 · --version
```

La pantalla

La distribución es fija: una **barra de estado** arriba con el estado, el nombre del PCE, la carpeta de la corrida y el contador de pasos; una **barra lateral** a la izquierda con los diez pasos y su estado (✓ hecho, ▶ activo, ○ pendiente, ✗ fallido, – omitido) y los eventos recientes; el **panel** del paso activo a la derecha (tablas que se recorren con las flechas, detalle "propuesto frente a en el PCE" lado a lado, barras de progreso); el **panel de log** abajo con la salida de workloader en vivo, cada comando repetido con sus argumentos; y una **barra de teclas** que cambia por paso. Cada decisión que importa es un modal: modales de elección (una tecla por opción, `esc` cancela), formularios (`tab` entre campos, `enter` envía) y cuadros de información. Las confirmaciones destructivas tienen el título en rojo. La interfaz está en inglés.

UMWL-TUI · PASO 4 (RECONCILIAR POR IP)

umwl-tui ● Ready

PCE default (pce.yaml) | run 20260903-042451 | step 4/10

STEPS

- ✓ 0 Preflight
- ✓ 1 Load CSV
- ✓ 2 PCE inventory
- ✓ 3 Labels
- ▶ 4 Reconcile by IP
- 5 Review new
- 6 Dry run
- 7 Execute
- 8 Verify
- 9 IP lists
- 10 Report

EVENTS

04:24 reconcile: 21 NEW...

■ Reconcile by IP – 24 rows · 3 awaiting a decision

IP	PROPOSED NAME	STATE	IN THE PCE
10.10.4.21	Zabbix Server 10.10.4...	EXISTS-UNMANAGED	zbx-olD
10.30.4.21	Zabbix Proxy Cloud 10...	NEW	
10.10.0.53	DNS Corporativo 10.10...	CONFLICT-MULTIPLE	dns1

■ Selected: 10.10.4.21

proposed		in the PCE · unmanaged	
name	Zabbix Server 10.10.4.21	name	Zabbix Server
role	Monitoring	role	–
app	Monitoring	app	–

workloader output · tab to scroll

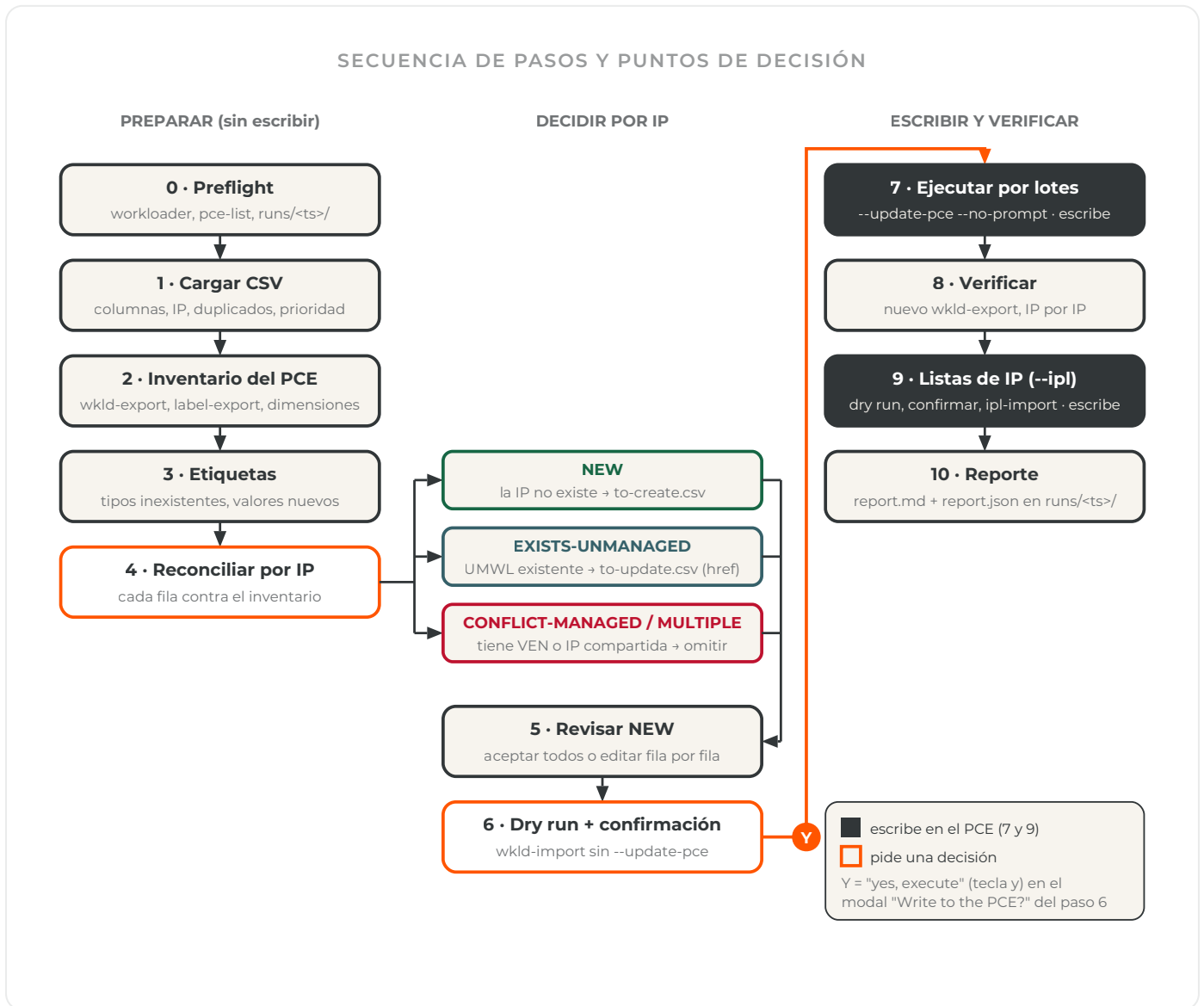
04:24:51 \$ workloader wkld-export --output-file runs/.../pce-workloads.csv

u update s skip c create anyway r rename+update U/S all of this kind enter next tab log ? help q quit

Teclas globales. `tab` pasa el foco al panel de log (se recorre con las flechas); `?` abre la ayuda; `q` sale, con confirmación una vez que empezó el trabajo; `ctrl+c` sale de inmediato. Las tablas aceptan `↑ ↓ j k pgup pgdown g G`. Los modales capturan todas las teclas mientras están abiertos.

El selector de archivos. Sin un CSV como argumento, `umwl-tui` abre después del preflight un selector estilo Midnight Commander: un campo de ruta editable arriba (`tab` o `/` para editarlo, `~` se expande, `enter` sobre un directorio salta ahí), el listado abajo con `..` primero, después los directorios, después los archivos `.csv` resaltados y el resto atenuado; `←` o `backspace` va al padre, `enter` o `→` abre

o selecciona, `esc` cancela. Después pide el CSV opcional de listas de IP (`esc` = ninguno) y el filtro de prioridad. Un CSV que no pasa la validación muestra el motivo en un diálogo, vuelve al preflight y reabre el selector.



Qué hace y qué decide cada paso

PASO	QUÉ PASA	TECLAS
0 Preflight	Ubica workloader (flag → ruta guardada → <code>./workloader</code> → PATH) e imprime su versión y arquitectura de CPU, ofrece compilarlo desde el fuente o descargarlo, corre <code>pce-list</code> y muestra el PCE y la carpeta de trabajo, prueba la conexión con un <code>label-dimension-export</code> de solo lectura, crea <code>runs/<ts>/</code> . Las tres comprobaciones deben pasar (un binario Intel bajo Rosetta es advertencia, no falla).	<code>enter</code> siguiente · <code>b</code> compilar · <code>d</code> descargar · <code>w</code> ruta · <code>p</code> pce-add · <code>t</code> probar · <code>r</code> revisar de nuevo

1 Load CSV	Columnas requeridas (<code>hostname</code> , <code>name</code> , <code>interfaces</code>), IP válidas, IP repetidas, el filtro <code>--priority</code> , hostnames o nombres duplicados (se agrega la IP como sufijo), columnas de etiqueta encontradas. Las advertencias y las filas descartadas se listan arriba de la tabla.	<code>enter</code> inventario · <code>↑↓</code> recorrer · <code>o</code> otro CSV
2 PCE inventory	<code>wkld-export</code> , <code>label-export</code> , <code>label-dimension-export</code> a <code>runs/<ts>/</code> ; cada workload del PCE se indexa por sus IP (<code>interfaces</code> y <code>public_ip</code>) y se marcan los gestionados (<code>managed</code> o <code>ven_href</code>). Solo lectura; sigue solo.	—
3 Labels	Las columnas del CSV que no son tipo de etiqueta de este PCE se señalan: descartarlas o salir y crear el tipo con <code>label-dimension-import</code> . Se listan y confirman los valores que workloader va a crear.	<code>enter</code> aceptar · <code>d</code> descartar columnas desconocidas
4 Reconcile by IP	Cada fila se clasifica: NEW (la IP no existe), EXISTS-UNMANAGED (la IP pertenece a un UMWL), CONFLICT-MANAGED (la IP pertenece a un workload con VEN), CONFLICT-MULTIPLE (varios workloads comparten la IP). El detalle muestra la fila propuesta junto al objeto del PCE. Actualizar un workload gestionado pide una confirmación extra porque puede cambiar su política de inmediato; <code>U</code> nunca toca las filas CONFLICT-MANAGED. Las filas sin decidir al pulsar <code>enter</code> se omiten tras un modal.	<code>U</code> actualizar · <code>S</code> omitir · <code>C</code> crear igual · <code>R</code> renombrar + actualizar · <code>U / S</code> todas las del mismo tipo · <code>n</code> siguiente sin decidir · <code>enter</code> siguiente
5 Review new	Tabla de los workloads a crear, con los conteos de actualizar y omitir; editar name, hostname, description y etiquetas fila por fila, u omitir una fila. Escribe <code>to-create.csv</code> , <code>to-update.csv</code> y <code>skipped.csv</code> .	<code>enter</code> aceptar todo → dry run · <code>e</code> editar · <code>s</code> omitir
6 Dry run	<code>wkld-import to-create.csv --umwl --update=false --match name</code> y <code>wkld-import to-update.csv --match href</code> , ambos sin <code>--update-pce</code> ; se muestran las líneas relevantes del log. <code>enter</code> abre el modal rojo "Write to the PCE?" con los conteos; solo <code>y</code> escribe.	<code>enter</code> ejecutar · <code>x</code> parar + reporte · <code>↑↓</code> desplazar
7 Execute	Lotes de <code>--chunk</code> filas, una llamada a <code>wkld-import ... --update-pce --no-prompt</code> por lote, primero los lotes de creación y después los de actualización, con barra de progreso y la primera y última IP de cada lote. Un lote fallido abre un modal: reintentar, saltar (queda registrado) o abortar.	modal: <code>r</code> reintentar · <code>s</code> saltar · <code>q</code> abortar
8 Verify	Un nuevo <code>wkld-export</code> ; cada IP creada debe aparecer en el inventario. Muestra creados, actualizados, etiquetas creadas y lotes fallidos.	<code>enter</code> siguiente
9 IP lists	Solo con <code>--ipl</code> : dry run de <code>ipl-import</code> , después un modal de confirmación, después <code>ipl-import ... --update-pce --no-prompt</code> .	<code>enter</code> importar · <code>n</code> omitir
10 Report	<code>runs/<ts>/report.md</code> y <code>report.json</code> : estado, creados, actualizados, omitidos, etiquetas creadas, lotes fallidos, verificación y cada comando de workloader con su código de salida.	<code>enter</code> o <code>q</code> terminar

LEER EL DRY RUN: "THE MATCH COLUMN CANNOT BE BLANK" Y "NOTHING TO BE DONE"

workloader hace match de cada fila por una sola columna, elegida con `--match` o, sin él, por prioridad entre las columnas presentes en el CSV: `href`, después `hostname`, después `name`. Una fila cuya columna de match está vacía se salta con "the match column cannot be blank"; no hay respaldo de `hostname` a `name`, y un CSV en el que se saltan todas las filas termina con "nothing to be done"³. Con la convención de `hostname` vacío de la sección 7, un `wkld-import` a secas elegiría `hostname` y saltaría todas las filas. Por eso `umwl-tui` pasa `--match name` en la pasada de creación siempre que alguna fila tenga `hostname` vacío y `--match href` en la pasada de actualización, y por eso un dry run que contenga cualquiera de esos mensajes, un "[WARN]" o un "[ERROR]" se reporta como no ok: el modal "Write to the PCE?" lo dice. Un lote que no hace nada es una corrida perdida; una fila saltada es una fila que el PCE no va a recibir.

TODO LO QUE PRODUJO LA CORRIDA

`runs/<timestamp>/` contiene los inventarios del PCE antes y después, `to-create.csv`, `to-update.csv`, `skipped.csv`, un CSV y un log de workloader por lote (`create-chunk001.csv`, `create-chunk001.log` ...), los logs del dry run, `report.md` y `report.json`. Las carpetas de corrida son 0755 y los CSV 0644: contienen IP del cliente pero ningún secreto. Salir en el dry run con `x` igual escribe el reporte, marcado "stopped before writing to the PCE".

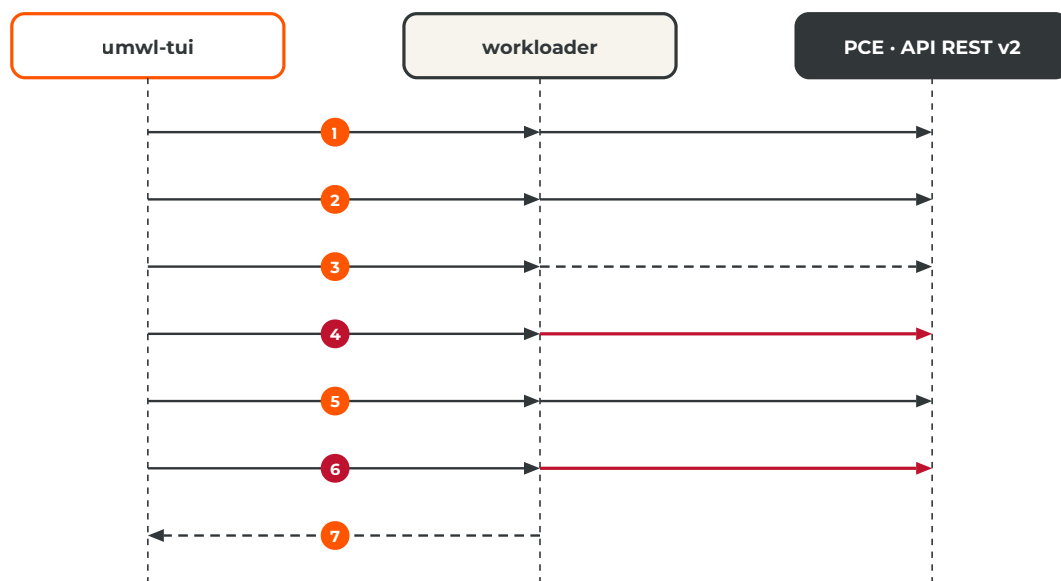
FUENTES — DOCUMENTACIÓN OFICIAL

1. workloader — README: `--update-pce` y `--no-prompt` ("auto-accept this prompt, as would be needed in automation"); logs en `workloader.log` — github.com/brian1917/workloader
2. workloader — `cmd/root.go`: flags persistentes `--pce`, `--log-file`, `--update-pce`, `--no-prompt`, `--config-file` — github.com/brian1917/workloader/blob/master/cmd/root.go
3. workloader — `cmd/wkldimport/cmd.go`: `--match (href, hostname, name, external_data)`, prioridad de la columna de match, "the match column cannot be blank", `--umwl`, `--update` — github.com/brian1917/workloader/blob/master/cmd/wkldimport/cmd.go

■ umwl-tui y workloader

`umwl-tui` nunca habla directamente con la API del PCE: ejecuta subcomandos de `workloader` (el binario `./workloader` de la carpeta de trabajo, o el del PATH, o el de `--workloader`) con `exec` y argumentos tipados, sin ningún shell en el medio, y lee sus logs. Cada llamada se ejecuta con `--log-file runs/<ts>/<paso>.log` y, si se indicó, con `--pce <nombre>`. Sin `--update-pce`, ningún comando de `workloader` modifica el PCE¹; la aplicación agrega ese flag (junto con `--no-prompt`) solo en los pasos 7 y 9, después del modal de confirmación explícito.

QUIÉN LLAMA A QUIÉN: UMWL-TUI, WORKLOADER Y PCE



Trazo rojo = escritura en el PCE (solo tras confirmar). Trazo punteado = solo lectura / log.

1. `pce-list` (y `pce-add --api-key ...` desde el formulario del preflight cuando no hay ningún PCE configurado); la prueba de conexión es un `label-dimension-export`.
2. `wkld-export`, `label-export`, `label-dimension-export` a `runs/<ts>/`: inventario (con `managed`, `ven_href`, `interfaces`, `public_ip`), etiquetas y tipos. Solo lectura.
3. Dry run: `wkld-import to-create.csv --umwl --update=false --match name` y `wkld-import to-update.csv --match href`, sin `--update-pce`; `workloader` registra el plan en el log y no modifica nada.
4. Por lote: los mismos dos comandos sobre `create-chunkNNN.csv` / `update-chunkNNN.csv` más `--update-pce --no-prompt`. Crea UMWL, actualiza etiquetas y descripción de los existentes y crea los valores de etiqueta que falten.
5. `wkld-export` de nuevo: cada IP creada debe aparecer en el inventario.
6. Con `--ipl`: `ipl-import archivo.csv` (dry run) y, tras confirmar, `ipl-import archivo.csv --update-pce --no-prompt`.
7. Cada llamada deja su log en `runs/<ts>/`; las líneas "to be created", "to be changed", "created new ... label", "bulk create/update workload successful", "[WARN]" y "[ERROR]" se muestran en pantalla y se conservan en el reporte.

Semántica de la actualización

Una fila decidida como *actualizar* se reescribe antes del dry run con el `href` del objeto del PCE, el `hostname` del PCE, `interfaces` en blanco (workloader deja las interfaces sin tocar cuando la celda está vacía) y el `name` del PCE salvo que se haya renombrado. Eso es lo que hace que un `wkld-import --match href` a secas (sin `--umwl`) actualice solo etiquetas y descripción.

Los mismos pasos sin umwl-tui

TERMINAL · EQUIVALENTES MANUALES (MISMA CARPETA DE TRABAJO, MISMO PCE.YAML)

```
# PCE (una vez por cuenta + PCE) e inventario
./workloader pce-add --api-key --name cliente-a --fqdn pce.cliente-a.example --port 443 \
  --api-user api_xxx --api-secret '...' --org 1
./workloader wkld-export --output-file pce-workloads.csv
python3 kit/legacy/reconcile_umwl.py pce-workloads.csv cliente-a-umwl-import.real.csv

# dry run (leer workloader.log), después escribir — los hostnames vacíos necesitan --match name
./workloader wkld-import cliente-a-umwl-import.real-to-create.csv --umwl --match name
./workloader wkld-import cliente-a-umwl-import.real-to-create.csv --umwl --match name --update-pce

# existentes: solo etiquetas y descripción, match por href
./workloader wkld-import cliente-a-umwl-import.real-existing.csv --match href --update-pce
./workloader ipl-import cliente-a-ipl-import.csv --update-pce
```

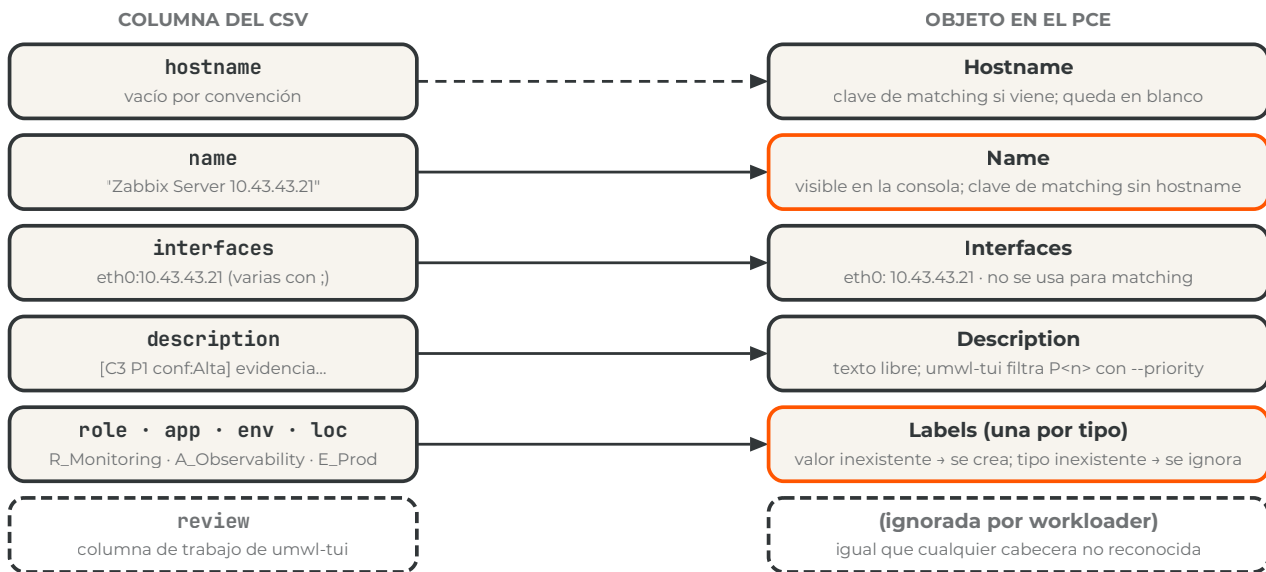
PCE-ADD NECESITA --API-KEY PARA USAR UNA API KEY

En el código de workloader, `--api-user`, `--api-secret` y `--org` solo se leen cuando está presente el flag `--api-key`; sin él, el comando pide correo y contraseña². El formulario del preflight de `umwl-tui` siempre lo agrega.

Del CSV a los objetos del PCE

`wkld-import` reconoce un conjunto fijo de cabeceras (`href`, `hostname`, `name`, `interfaces`, `public_ip`, `description`, `enforcement`, `visibility` y otras) más **una columna por tipo de etiqueta existente**; el resto se ignora³. El diagrama muestra a qué campo del workload va cada columna.

COLUMNAS DEL CSV Y CAMPOS DEL WORKLOAD NO GESTIONADO



Matching: por href, hostname o name, nunca por IP. Antes de crear, workloader busca si la fila ya existe por una sola columna: `href` si la columna viene, si no `hostname`, si no `name`, o la que se fuerce con `--match`³. La IP no participa. Por eso dos filas con el mismo `name` y sin hostname son el mismo workload para workloader (el paso 1 les agrega la IP como sufijo), y por eso un UMWL que ya existe con otro nombre se crearía duplicado si no se reconcilia antes por IP contra el `wkld-export`: ese es el paso 4 de `umwl-tui` y el único trabajo de `legacy/reconcile_umwl.py`.

Listas de IP. El contrato de `ipl-import` es `name,description,include,exclude,fqdns`: `name` es la clave de matching cuando no hay `href` (si existe, actualiza; si no, crea)⁴ y por convención va en minúsculas con prefijo `ipl-`; `include` y `exclude` llevan IP, CIDR o rangos separados por `;`; `fqdns`, nombres cuando corresponde; `description`, el razonamiento del informe.

FUENTES — DOCUMENTACIÓN OFICIAL

- workloader — README: "When a command modifies resources in the PCE, workloader does not trigger the action unless the `--update-pce` flag is included" — github.com/brian1917/workloader
- workloader — `cmd/pcemgmt/addpce.go`: flags de `pce-add` (`--api-key`, `--api-user`, `--api-secret`, `--org`, `--disable-tls-verification`) — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
- workloader — `cmd/wkldimport/cmd.go`: cabeceras reconocidas, formato de interfaces, `--umwl`, `--update`, `--match` (`href`, `hostname`, `name`, `external_data`) — github.com/brian1917/workloader/blob/master/cmd/wkldimport/cmd.go
- workloader — `cmd/iplimport/iplimport.go`: cabeceras `name`, `description`, `include`, `exclude`, `fqdns`; separador `;"`; matching por `href` y `name` — github.com/brian1917/workloader/blob/master/cmd/iplimport/iplimport.go

■ Iterar: exports v2

Después de aprovisionar las primeras reglas, exportá de nuevo con el mismo alcance y una ventana nueva (sección 4), anonimízalo con el mismo `anon-map.json` (sección 5) y corré la variante v2 de `prompt-short.md` en una conversación nueva del mismo Proyecto, con el informe anterior y los CSV ya cargados o propuestos adjuntos. El informe v2 compara ventana, filas y ruido con el export anterior, muestra qué pasó a Allowed y qué sigue en No Rule, lista los peers que no se vieron en la ventana nueva (marcados "sin tráfico en la ventana nueva", nunca borrados) y vuelve a entregar los CSV completos, no solo el delta, con `review` = PENDING para las filas nuevas, UPDATED para las filas cuyo comentario o etiquetas cambiaron y UNCHANGED para el resto, más una tabla de cambios (IP, antes, después, motivo) y una sección de hallazgos actualizada.

Después de `deanon`, `umwl-tui` hace el resto: en el paso 4 las filas cuya IP ya existe salen como EXISTS-UNMANAGED y se actualizan por `href` (etiquetas y descripción); las nuevas se crean. Volver a cargar el CSV completo, por lo tanto, solo cambia lo que cambió. Nombrá la conversación y los CSV con la versión (`<grupo>-umwl-import-v2.csv`) y conservá las carpetas `runs/` de ambas cargas en la carpeta del cliente.

FUENTES — REPOSITORIO

1. [illumio-workloader-import-kit — docs/prompts/context.md](#) (procedimiento v2) y [docs/prompts/prompt-short.md](#) (variante v2); [examples/sample-umwl-import-v2.csv](#) — [github.com/roschereric/illumio-workloader-import-kit](#)

■ Solución de problemas y lista de control

SÍNTOMA	CAUSA Y SOLUCIÓN
macOS no abre <code>umwl-tui</code> o <code>workloader</code> ("no se pudo verificar el desarrollador")	Cuarentena de Gatekeeper sobre un archivo descargado. <code>xattr -d com.apple.quarantine ./umwl-tui</code> (lo mismo para <code>./workloader</code>) y <code>chmod +x</code> . El preflight lo hace para workloader cuando lo descarga.
La fila del chequeo dice <code>amd64 binary on arm64 (runs under emulation/Rosetta 2)</code>	Tenés el release de macOS upstream, que es solo Intel. Presioná <code>b</code> en el preflight para clonar y compilar un binario nativo (requiere <code>brew install go</code>), o seguí el bloque c' de la sección 3. Rosetta sigue funcionando mientras tanto (<code>softwareupdate --install-rosetta</code> si falta). <code>umwl-tui</code> sí tiene builds nativos arm64 y amd64.
<code>b</code> dice "Go toolchain missing"	<code>brew install go</code> (el diálogo ofrece ejecutarlo cuando hay Homebrew) o el instalador de go.dev/dl ; después <code>r</code> y <code>b</code> de nuevo. Sin Go, <code>d</code> descarga el release.
<code>go build</code> falla con <code>proxy.golang.org ... 403</code> o "Host not in allowlist"	La red filtra el proxy de módulos de Go. Desde <code>workloader-src/</code> ejecutá <code>GOPROXY=direct go build ...</code> (los módulos vienen directo de GitHub), o compilá en una máquina con salida abierta y copió el archivo: es un único binario estático.

umwl-tui usa un workloader distinto del esperado	Precedencia: --workloader → ./umwl-tui.json → ~/.config/umwl-tui/config.json → ./workloader → PATH. La fila del chequeo imprime "[path from ...]" cuando lo aportó un archivo de configuración; w lo cambia, borrar el archivo lo resetea.
El dry run termina con "the match column cannot be blank" en todas las filas y "nothing to be done"	workloader hizo match por hostname y todos los hostnames están vacíos (sección 8). umwl-tui pasa --match name en la pasada de creación y --match href en la de actualización, y reporta ese dry run como no ok. A mano: wkld-import to-create.csv --umwl --match name .
El selector de archivos no muestra mi CSV	Lista la carpeta actual: escribí la ruta en el campo de arriba (tab o /), usá .. para subir, o pasá el CSV en la línea de comandos. Los archivos con otra extensión aparecen atenuados pero se pueden seleccionar.
"Load CSV" falla y el selector se vuelve a abrir	Faltan columnas requeridas (hostname , name , interfaces), hay una IP inválida en interfaces o ninguna fila coincide con --priority . El motivo se muestra en un diálogo y en el panel de log; corregí el CSV y volví a elegirlo desde el selector (o pulsá o en el paso 1 para elegir otro archivo).
pce-add pide correo y contraseña	Se ejecutó workloader pce-add a mano sin --api-key ; sin ese flag workloader ignora --api-user , --api-secret y --org e intenta un login con usuario y contraseña. El formulario del preflight siempre agrega --api-key .
El dry run funciona y el import devuelve 401/403	La API key hereda el rol del usuario. Hace falta un rol global con escritura (Global Organization Owner o Global Administrator; un Workload Manager con alcance puede crear workloads pero no etiquetas nuevas ni listas de IP) o una service account equivalente. Revisar también el org id .
pce-list muestra otro cliente u otra organización	Carpeta equivocada (el directorio actual decide qué pce.yaml se usa) o ILLUMIO_CONFIG apuntando a otro archivo. Una carpeta por cuenta + PCE; unset ILLUMIO_CONFIG ; --pce <nombre> solo si el pce.yaml tiene varios perfiles.
El dry run dice que va a actualizar un workload que no esperabas	Match por name con un objeto existente. Revisá runs/<ts>/dry-create.log ; renombrá la fila en el paso 5 o dejá que la reconciliación por IP del paso 4 lo resuelva.
Se cargó un solo workload de varios con el mismo nombre	Mismo hostname (o mismo name sin hostname) = mismo workload para workloader. Hacé único el name ; el paso 1 agrega la IP como sufijo.
Una columna de etiqueta no se aplicó (falta el tipo de etiqueta)	El tipo no existe en el PCE y wkld-import ignora la columna; el paso 3 lo señala. Crealo en la consola (Settings › Label Settings) o con ./workloader label-dimension-import tipos.csv --update-pce (CSV con key y display_name) ¹ y volví a correr.
wkld-import crea una etiqueta con otras mayúsculas	Los valores distinguen mayúsculas salvo con --ignore-case . Usá exactamente los valores que muestra label-export (adjuntalo al análisis, sección 4.2).
Lote fallido en el paso 7	Revisá runs/<ts>/create-chunkNNN.log ; el CSV de ese lote queda en create-chunkNNN.csv para reintentarlo desde el modal o a mano con wkld-import ... --umwl --match name --update-pce .
El export de tráfico tiene exactamente N filas redondas	Está truncado al límite de la consulta. Subí el límite en Results Settings, filtrá por aplicación, excluí escáneres, acortá la ventana y volví a exportar; la consola muestra hasta 100.000 resultados y el CSV hasta 200.000 ^{2,3} .

El anonimizador advierte sobre una fuga

Sobrevivió un dominio real, el nombre del cliente o un hostname corto (a menudo dentro de un valor de etiqueta o una descripción). Agregalo con `--customer` o `--domain` y corré `anon` de nuevo con el mismo mapa.

Error TLS con un PCE on-prem

Certificado interno. Instalá la CA en el llavero; el campo "Disable TLS verify" del formulario del preflight (`--disable-tls-verification true`) solo en laboratorio.

Antes de escribir en el PCE

- ☐ **Carpeta correcta** — el directorio actual es la carpeta de la cuenta + PCE, el preflight muestra ese PCE (FQDN y org) y no hay `ILLUMIO_CONFIG` definida.
- ☐ **Nombres restaurados** — se corrió `deanon` sobre los CSV propuestos y las descripciones muestran FQDN reales.
- ☐ **CSV en el formato v2** — hostname vacío, `name` único con IP, `interfaces` como `eth0:<ip>`, etiquetas con prefijo, prioridad en la descripción.
- ☐ **Tipos de etiqueta** — cada columna de etiqueta del CSV existe en el PCE (paso 3 sin advertencias).
- ☐ **Reconciliación revisada** — ningún CONFLICT-MANAGED se actualiza sin haberlo mirado; los EXISTS-UNMANAGED conservan el nombre del PCE salvo renombrado explícito.
- ☐ **Dry run leído** — las líneas "to be created" coinciden con lo esperado; no hay "cannot be blank", "nothing to be done", "[WARN]" ni "[ERROR]".
- ☐ **Lote pequeño la primera vez** — `--chunk 5` en la primera corrida contra un PCE nuevo para acotar el impacto de un error.
- ☐ **Reporte archivado** — `runs/<ts>/report.md` queda en la carpeta del cliente; no se sube al repositorio ni se adjunta a una conversación.

FUENTES — DOCUMENTACIÓN OFICIAL

1. [workloader](https://github.com/brian1917/workloader/blob/master/cmd/labeldimension/import.go) — `cmd/labeldimension/import.go`: `label-dimension-import` (cabeceras `key`, `display_name`, `href...`) — github.com/brian1917/workloader/blob/master/cmd/labeldimension/import.go
2. Illumio Core 25.4 — Visualization — About the Visualization Tools (Explore, Traffic, límites de resultados, consultas asíncronas) — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Visualization/out/en/visualization-tools/about-the-visualization-tools.html
3. Illumio Core 22.5 — REST API — Explorer: `traffic_flows/async_queries`, `max_results 200.000` — product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html

■ Apéndice A — Columnas del export

Cabecera del export de Traffic de una consola Illumio Core 24.x/25.x (la misma línea está en `docs/export-columns.txt`, pensada para el conocimiento del proyecto). Los tipos de etiqueta personalizados aparecen como columnas adicionales `Source <Tipo>` / `Destination <Tipo>`¹.

DOCS/EXPORT-COLUMNS.TXT · CABECERA DEL EXPORT DE TRAFFIC

```
Source IP, Source IPIList, Source Name, Source Hostname, Source Enforcement,
Source Application, Source Environment, Source Location, Source Quarantine, Source Role,
Source Service Category, Source Service Role, Source FQDN, Source CSP ID, Source Account ID,
Source Tenant ID, Source Detail Type, Source Object Type, Source Category, Source Sub Category,
Source Region, Consuming Process, Consuming Service, Consuming Username,
Destination IP, Destination IPIList, Destination Name, Destination Hostname, Destination Enforcement,
Destination Application, Destination Environment, Destination Location, Destination Quarantine,
Destination Role, Destination Service Category, Destination Service Role, Destination FQDN,
Transmission, Destination CSP ID, Destination Account ID, Destination Tenant ID,
Destination Detail Type, Destination Object Type, Destination Category, Destination Sub Category,
Destination Region, Port, Protocol, Process, Service, Username, Num Flows, Bytes In, Bytes Out,
Connection State, Reported Policy Decision, Reported by, First Detected, Last Detected
```

GRUPO DE COLUMNAS	QUÉ LEE EL ANÁLISIS DE AHÍ
Source / Destination IP, Name, Hostname, FQDN	La identidad del peer. Las IP privadas son la clave de unión; nombres y FQDN se seudonimizan antes del análisis.
Source / Destination Enforcement y columnas de etiquetas	Qué lado tiene VEN (un valor en Enforcement) y sus etiquetas; una columna por tipo de etiqueta.
Consuming Process / Service / Username, Process, Service, Username	Evidencia de rol del lado del VEN: qué proceso abrió o sirvió la conexión y con qué usuario.
Port, Protocol, Transmission	El servicio y si el tráfico es unicast, broadcast o multicast.
Num Flows, Bytes In, Bytes Out, First / Last Detected	Volumen y patrón temporal (continuo, programado, puntual).
Reported Policy Decision, Connection State, Reported by	Allowed, Blocked o Potentially Blocked según lo reportado, y qué lado reportó el flujo.

FUENTES — DOCUMENTACIÓN OFICIAL

1. Illumio Core 24.2 — Visualization — Traffic Table (Export to CSV, una columna por tipo de etiqueta) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Visualization/out/en/visualization-tools/traffic-table.html

■ Apéndice B — Archivos del repositorio

ARCHIVO	PROPÓSITO
<code>umwl-tui</code> (<code>cmd/umwl-tui</code> , <code>internal/</code> , <code>Makefile</code>)	La aplicación de pantalla completa (Go, Bubble Tea): código fuente, build (<code>make build</code>), pruebas (<code>make test</code>), compilación cruzada con checksums (<code>make dist</code>). Los binarios precompilados van adjuntos a los releases de GitHub.
<code>anonymize_export.py</code>	Seudonimizar exports (<code>anon</code>) y restaurar nombres en las propuestas (<code>deanon</code>). Biblioteca estándar de Python.
<code>docs/prompts/context.md</code>	Instrucciones del proyecto: el método completo y los contratos de los entregables.
<code>docs/prompts/prompt-short.md</code>	El mensaje por conversación (variantes v1 y v2).
<code>docs/export-columns.txt</code> , <code>docs/img/*.svg</code>	Línea de cabecera de un export de tráfico (Apéndice A); el diagrama del flujo y los esquemas de la consola de la sección 4.
<code>docs/SPEC-umwl-tui.md</code> , <code>CLAUDE.md</code>	El contrato de la aplicación (máquina de estados, UI, contrato del motor con workloader, requisitos de seguridad, backlog) y las notas de build, pruebas y release para quien la extienda.
<code>legacy/umwl_loader.py</code>	El loader anterior de terminal plana (Python, biblioteca estándar): los mismos diez pasos, conservado como respaldo.
<code>legacy/reconcile_umwl.py</code>	Reconciliación por IP mínima y no interactiva: a partir de un <code>wkld-export</code> y el CSV propuesto escribe <code>-to-create.csv</code> , <code>-existing.csv</code> , <code>-conflicts.csv</code> y un reporte de texto. Nunca escribe en el PCE.
<code>examples/*-umwl-import*.csv</code> , <code>examples/*-ipl-import*.csv</code>	CSV de una prueba de concepto, una fila por IP; los archivos <code>-v2</code> son las referencias de formato (etiquetas con prefijo, name = rol + IP, hostname vacío, <code>eth0:<ip></code>). Solo plantillas: contienen las IP de ese laboratorio.
<code>.gitignore</code>	Excluye los binarios, <code>pce.yaml</code> , <code>anon-map.json</code> , <code>*.anon.csv</code> , <code>*.real.csv</code> , logs, <code>runs/</code> , <code>dist/</code> y zips.

FUENTES — REPOSITORIO

1. [illumio-workloader-import-kit — README, CLAUDE.md, docs/SPEC-umwl-tui.md, gitignore](#) — github.com/roschereric/illumio-workloader-import-kit

ACERCA DE ESTE DOCUMENTO

Esta guía fue preparada por Eric Roscher – Illumio LATAM Pre-Sales Engineering para explicar el uso de principio a fin de umwl-tui y del repositorio illumio-workloader-import-kit: exportar los flujos del PCE, anonimizarlos, correr el análisis con Claude, revisar las propuestas y cargar workloads no gestionados y listas de IP con workloader. Es un material de apoyo para el destinatario y no constituye una publicación oficial de Illumio, Inc.; no modifica ni reemplaza la documentación oficial del producto, los acuerdos contractuales ni los alcances de trabajo (SOW). Valide todos los detalles técnicos contra la documentación oficial correspondiente a sus versiones específicas antes de actuar sobre ellos. Illumio y el logotipo de Illumio son marcas de Illumio, Inc., utilizadas aquí para identificar el tema tratado.